

Video-to-3DGS Pipeline — Documentation

Overview

This project converts a video into a 3D Gaussian Splatting (3DGS) `.ply` model using a fully automated cloud pipeline.

Architecture at a glance:

User (Telegram) → Railway Bot → RunPod Serverless GPU → S3 → Telegram (PLY file)

1. A user sends a video to the Telegram bot
 2. The bot collects settings via a 5-step wizard, uploads the video to S3, and submits a job to RunPod
 3. RunPod runs the pipeline on a GPU worker: extract frames → SfM reconstruction → 3DGS training (Brush) → upload PLY to S3
 4. The bot polls for completion and delivers the `.ply` file back to the user
-

Repository Structure

```

gaussian-splatting/
├─ handler.py           # RunPod serverless entry point
├─ video_to_3dgs.py     # Full pipeline script (frames → SfM → 3DGS → S3)
├─ hloc_sfm.py          # HLoc SfM backend (SuperPoint + LightGlue + COLMAP)
├─ mast3r_sfm.py        # MAST3R SfM backend (deep matching + GLoMAP)
├─ Dockerfile           # Docker image for RunPod workers (~21 GB)
├─ example_inputs.json  # Job input examples for all 4 backends
├─ run_pipeline.sh       # Local pipeline runner script
├─ train.py             # 3DGS training script
├─ render.py            # Render script
├─ metrics.py           # Evaluation metrics
├─ full_eval.py         # Full evaluation runner
├─ convert.py           # Data conversion utilities
├─ arguments/           # Argument parsing modules
├─ gaussian_renderer/   # Gaussian rendering modules
├─ scene/               # Scene representation modules
├─ utils/               # Utility functions
├─ lpipsPyTorch/        # LPIPS perceptual loss
├─ submodules/          # Git submodules (diff-gaussian, simple-knn, etc.)
├─ LocalVSServerless/   # Benchmark comparison outputs
├─ telegram_bot/
│   └─ bot.py           # Full Telegram bot (single file)
│   └─ Procfile         # Railway process definition

```

```

├── requirements.txt # Bot Python dependencies
├── PROJECT_STATUS.md # Living project log (sessions, bugs, decisions)
└── .env # Local credentials (gitignored – never commit)

```

Pipeline Steps

When a job is submitted, `video_to_3dgs.py` runs these steps in sequence:

Step	Description
Frame extraction	<code>ffmpeg</code> extracts frames at the configured FPS using software decode (not CUDA)
Frame selection	Sharpest frames selected per window using <code>oversample</code> factor
SfM reconstruction	Chosen backend reconstructs camera poses and a sparse point cloud
Undistortion	COLMAP undistorts images for Brush training
3DGS training	Brush trains the Gaussian splat model
PLY upload	Result <code>.ply</code> uploaded to S3 at <code>3dgs-outputs/<job_id>/</code>
Timings upload	<code>timings.txt</code> uploaded alongside the PLY

SfM Backends

Backend	Feature extractor	Mapper	Best for	Default FPS	Unique arg
<code>mast3r</code>	MASt3R (deep)	GLOMAP	Highest quality, challenging scenes	1	<code>window_size</code> (default 10)
<code>fastmap</code>	SIFT	FastMap	Speed, good default	2	<code>match_overlap</code> (default 5)
<code>colmap</code>	SIFT	COLMAP incremental	Tricky scenes, reliability	2	<code>match_overlap</code> (default 5)
<code>hloc</code>	SuperPoint + LightGlue	COLMAP	Low-texture, metallic, reflective	2	<code>match_overlap</code> (default 5)

Job Input Parameters

Submitted as JSON to the RunPod endpoint:

Parameter	Type	Default	Description
<code>video_url</code>	string	required	HTTP/HTTPS URL or Google Drive share link
<code>sfm_backend</code>	string	<code>mast3r</code>	One of: <code>mast3r</code> , <code>fastmap</code> , <code>colmap</code> , <code>hloc</code>
<code>fps</code>	int	<code>2</code>	Frames per second extracted from the video
<code>iterations</code>	int	<code>7000</code>	Brush training steps (use <code>30000</code> for higher quality)
<code>max_resolution</code>	int	<code>1920</code>	Max image dimension in pixels passed to Brush
<code>oversample</code>	int	<code>3</code>	Extract <code>fps × oversample</code> frames, keep sharpest per window
<code>window_size</code>	int	<code>10</code>	[<code>mast3r</code> only] Sliding-window pair size for MAST3R matching
<code>match_overlap</code>	int	<code>5</code>	[<code>fastmap/colmap/hloc</code> only] COLMAP sequential matching overlap

VRAM guidance: For videos longer than ~60s or resolution above 1080p, use `fps=1` and `max_resolution=1280` to avoid GPU OOM crashes.

Docker Image

- **Registry:** `hjeij2000/video-to-3dgs`
- **Current stable tag:** `:v11` (2026-03-01)
- **Image size:** ~21 GB
- **Base:** `nvidia/cuda:12.4.1-devel-ubuntu22.04` (requires driver ≥ 550)
- **Key env vars set in image:**
 - `NVIDIA_DRIVER_CAPABILITIES=all` — required for Vulkan/wgpu (Brush)
 - Vulkan ICD configured via `/usr/share/vulkan/icd.d/nvidia_icd.json`

Build & Push Procedure

Use plain `docker build`, **NOT** `docker buildx build --push` — buildx intermittently loses DNS inside its network namespace.

```
# Step 1: Build (host network – no DNS issues)
docker build \
  --file gaussian-splatting/Dockerfile \
  --tag hjeij2000/video-to-3dgs:serverless \
  .

# Step 2: Push
docker push hjeij2000/video-to-3dgs:serverless

# Step 3: Create versioned tag server-side (increment vN)
docker buildx imagetools create hjeij2000/video-to-3dgs:serverless \
  --tag hjeij2000/video-to-3dgs:vN

# Step 4: Remove local image to free disk (~21 GB)
docker image rm hjeij2000/video-to-3dgs:serverless

# Also clear build cache after every build
docker builder prune -f
```

Image Version History

Tag	Date	Status	Notes
:v11	2026-03-01	✓ Current	Fixed <code>RUST_BACKTRACE=1 SIGSEGV</code> ; Brush retry (3 attempts); env vars on template
:v10	2026-02-28	⚠ Broken	<code>RUST_BACKTRACE=1</code> still present
:v9	2026-02-28	⚠ Broken	Introduced <code>RUST_BACKTRACE=1</code> ; wiped template env vars
:v8	2026-02-27	✓	CUDA 12.4.1 base; stderr deadlock fix; torch seeds
:v7	2026-02-27	✓	handler.py S3 regional endpoint fix
:v6	earlier	✓	Added <code>libopenblas0-pthread</code> for GLOMAP
:v5	earlier	✓	Fixed <code>ENTRYPOINT</code> overwritten by <code>docker commit</code>
:v4	earlier	✓	Added <code>libboost</code> for GLOMAP

RunPod Setup

Setting	Value
API domain	<code>https://api.runpod.ai</code> (NOT <code>.io</code>)
Endpoint ID	<code>uupefx2whvkg13</code> (name: <code>video-to-3dgs</code>)

Setting	Value
Template ID	mrgxwb470f (name: video-to-3dgs-v11)
GPU types	ADA_24 + AMPERE_24
Workers	0 min / 2 max, idle timeout 10 min, FlashBoot OFF

Template Env Vars

The RunPod template must include all 4 AWS vars. **Omitting "env" or passing "env": [] in saveTemplate will wipe them**, causing all jobs to fail with `KeyError: 'AWS_S3_BUCKET'`.

```
AWS_ACCESS_KEY_ID    = <key>
AWS_SECRET_ACCESS_KEY = <secret>
AWS_S3_BUCKET        = splats-bucket
AWS_S3_REGION        = me-south-1
```

Useful API Calls

```
# Submit a job
curl -X POST "https://api.runpod.ai/v2/$RUNPOD_ENDPOINT_ID/run" \
  -H "Authorization: Bearer $RUNPOD_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{
    "input": {
      "video_url": "https://example.com/video.mp4",
      "sfm_backend": "mast3r",
      "fps": 1,
      "iterations": 7000,
      "max_resolution": 960
    }
  }'
```

```
# Check job status
curl "https://api.runpod.ai/v2/$RUNPOD_ENDPOINT_ID/status/{JOB_ID}" \
  -H "Authorization: Bearer $RUNPOD_API_KEY"
```

```
# Download PLY from S3 (pre-signed URLs have region issues – use CLI instead)
AWS_ACCESS_KEY_ID=... AWS_SECRET_ACCESS_KEY='...' \
  aws s3 cp s3://splats-bucket/3dgs-outputs/{JOB_ID}/export_1000.ply ./export_1000.ply \
  --region me-south-1
```

Telegram Bot

User Flow

1. User sends a video file (≤ 50 MB), Google Drive share link, or direct video URL
2. Bot presents a 5-step wizard:
 - **Step 1:** SfM backend (`mast3r` / `fastmap` / `colmap` / `hloc`)
 - **Step 2:** FPS (frames per second to extract)
 - **Step 3:** Max resolution (pixels)
 - **Step 4:** Backend-specific arg (`window_size` or `match_overlap`)
 - **Step 5:** Iterations (Brush training steps)
3. Bot uploads the video to S3 at `telegram-inputs/<uuid>/<filename>` and submits the RunPod job
4. Bot polls every 20 seconds, editing the status message live
5. On completion:
 - Sends a timings summary (per-step breakdown)
 - Delivers the `.ply` as a Telegram file attachment (≤ 45 MB)
 - Falls back to a presigned S3 link for larger files

Supported Input Types

- Telegram video upload (mp4, mov, avi, mkv) — max 50 MB (Telegram bot limit)
- Google Drive share links (auto-converted to direct download URL)
- Any direct HTTP/HTTPS video URL

Environment Variables

```
TELEGRAM_BOT_TOKEN=...          # from BotFather
RUNPOD_API_KEY=...
RUNPOD_ENDPOINT_ID=uupefx2whvkg13
AWS_ACCESS_KEY_ID=...
AWS_SECRET_ACCESS_KEY=...
AWS_S3_BUCKET=splats-bucket
AWS_S3_REGION=me-south-1
```

Railway Deployment

The bot runs as a Railway **worker** (no HTTP port needed):

1. Connect Railway project to `gaussian-splatting/telegram_bot/` directory
2. Railway detects `Procfile` → runs worker: `python bot.py`
3. Set all env vars in the Railway dashboard
4. Bot uses long-polling (`run_polling`) — no webhook configuration needed

Note: Bot sessions are in-memory only. A Railway restart loses all in-progress wizard sessions. Affected users receive a "session expired" message and must send their video again.

Bugs Fixed

#	Bug	Fix	Version
1	ffmpeg CUDA decode crash (<code>CUDA_ERROR_NO_DEVICE</code>)	Removed <code>-hwaccel cuda -c:v hevc_cuid</code> ; uses software decode	v1
2	<code>XDG_RUNTIME_DIR</code> not set (wgpu/Brush)	<code>env.setdefault("XDG_RUNTIME_DIR", "/tmp")</code> in <code>run_command()</code>	v1
3	Brush SIGSEGV at startup (no Vulkan)	<code>NVIDIA_DRIVER_CAPABILITIES=all</code> + Vulkan ICD JSON in Dockerfile	v1
4	Pre-signed S3 URL <code>IllegalLocationConstraintException</code>	Use regional <code>endpoint_url</code> in boto3 client	v3/v7
5	Brush OOM on high-res/long videos ⚠️	Workaround: use <code>fps=1</code> , <code>max_resolution=1280</code> . Auto-scaling not yet implemented	—
6	Stale worker image cache → Brush SIGSEGV	Create new versioned tag, update RunPod template to force re-pull	v2
7	GLOMAP <code>libboost</code> missing	Added boost libs to Dockerfile <code>apt-get</code>	v4
8	<code>docker commit</code> overwrites <code>ENTRYPOINT</code> (workers ran <code>apt-get</code> and exited)	Always pass <code>--change='ENTRYPOINT [...]'</code> to <code>docker commit</code>	v5
9	GLOMAP <code>libopenblas.so.0</code> missing	Added <code>libopenblas0-pthread</code> to Dockerfile	v6
10	<code>docker buildx</code> DNS failure in builder container	Use plain <code>docker build</code> instead of <code>docker buildx build</code>	—
11	Bot: aiohttp brotli decode error on RunPod API	Added <code>Accept-Encoding: gzip, deflate</code> header to all aiohttp calls	bot
12	Bot: Telegram Markdown parse error on underscores in arg names	Replace underscores with spaces before embedding in Markdown	bot
13	Bot: job polling silently dropped (never sent result)	Use <code>ctx.application.create_task()</code> instead of <code>asyncio.ensure_future()</code>	bot

#	Bug	Fix	Version
14	Container fails to start on RTX 4090 nodes with driver <570	Downgraded base to <code>cuda:12.4.1 ; torch to 2.6.0+cu124</code>	v8
15	Handler stderr pipe deadlock (worker stuck WORKING)	Redirect stderr to temp file instead of PIPE	v8
16	Result inconsistency across runs ⚠️	Added <code>torch.manual_seed(42)</code> to <code>hloc_sfm.py</code> and <code>mast3r_sfm.py</code>	v8
17	<code>RUST_BACKTRACE=1</code> causes all Brush jobs to SIGSEGV consistently	Removed <code>RUST_BACKTRACE=1</code> from <code>run_command()</code>	v11
18	<code>saveTemplate</code> with <code>"env": []</code> wipes all AWS credentials	Always include all 4 AWS env vars in every <code>saveTemplate</code> call	v11

Known Issues / Future Work

Issue	Description	Workaround
Brush OOM auto-scaling	Handler doesn't auto-reduce fps/resolution for long or high-res videos	User must manually set <code>fps=1</code> , <code>max_resolution=1280</code>
PLY delivery for large files (>45 MB)	Falls back to presigned S3 link which is difficult for non-technical users	Potential: public permanent URL, or upfront size warning
Bot sessions lost on restart	In-memory session dict — Railway restart wipes all active wizards	User must re-send video after restart

Benchmark Results

All tests run on RTX 4090 with `IMG_2188.MOV` (iPhone HEVC, 2816×1584, 57.9s).

Backend	FPS	Max Res	PLY Size	Total Time	SfM Time
<code>fastmap</code>	1	960px	121.3 MB	160.6s	~60s
<code>hloc</code>	2	960px	80.0 MB	100.5s	~40s
<code>mast3r</code>	1	960px		387.3s	

Backend	FPS	Max Res	PLY Size	Total Time	SfM Time
			130.0 MB		308s (58 images, 54,572 tracks)

Local hloc test (`10vptV4p430GIpKTpT8dQFdopDF6-7ej0` , 172 MB, 58s video):

Step	Time
Frame extraction	30.9s
SfM	89.2s
Undistort	6.1s
Brush (3000 steps)	36.3s
Total	162.5s
Output	20 MB, 86,214 points

Local Development

Prerequisites

The project requires a dedicated virtualenv — do **not** use system Python:

```
/home/ibrahim/gaussian-splatting/venv/bin/python3
```

The venv contains: `h5py` , `pycolmap` , `lightglue` , `torch` , `open3d` , and all other deps.

Loading Credentials

```
export $(grep -v '^#' gaussian-splatting/.env | xargs)
```

Running the Pipeline Locally

```
/home/ibrahim/gaussian-splatting/venv/bin/python3 video_to_3dgs.py \
  --video_url "https://..." \
  --sfm_backend hloc \
  --fps 2 \
  --iterations 3000 \
  --max_resolution 960 \
  --output_dir /tmp/3dgs_output
```

Storage

- Total disk: 192 GB
 - Docker image: ~21 GB (not stored locally — pull from Docker Hub when needed)
 - Build cache: ~35 GB (prune after every build with `docker builder prune -f`)
 - LocalVSServerless/ benchmark outputs: ~330 MB
 - S3 bucket `splats-bucket` (region `me-south-1`):
 - `telegram-inputs/` — videos uploaded by bot users
 - `3dgs-outputs/` — output PLY files and timings.txt per job
-

Next Steps

- [] End-to-end test `:v11` — verify Brush completes for both 3000 and 7000 training steps
- [] Auto-scale `fps` and `max_resolution` in `handler.py` based on video duration and resolution (fixes Bug #5 properly)
- [] Improve PLY delivery for files >45 MB (public URL or upfront size estimate)
- [] Persist bot sessions across Railway restarts (Redis or a lightweight DB)